



31 DE MAIO DE 2026

SISTEMA DE EMBARQUE DE PASSAGEIROS NUM AEROPORTO

COMUNICAÇÃO E SINCRONIZAÇÃO ENTRE PROCESSOS

RICARDO PULIDO (131393) E INÊS SANTOS (103477)
SISTEMAS OPERATIVOS, RSI, ESTGA 2025/2026
Trabalho Prático 2



Arquitetura da solução

Descrição da organização do sistema, processos foram criados e como eles se comunicam

A solução para o Aeroporto foi implementada com uma arquitetura modular e concorrente, de modo a simular realisticamente um sistema de embarque de passageiros. É composta por vários ficheiros que representam entidades/classes independentes do sistema, embora todos interligados através de mecanismos de sincronização adequados: **servidor**, **passageiros**, **agentes**, **portões** e **gerador de passageiros**. A comunicação entre estes componentes é feita através de memória partilhada, PriorityQueue, multiprocessing, Queue, locks e processos concorrentes.

Organização do Sistema

Na pasta Server, incluímos os ficheiros: airport_server.py, agente_process.py, gate.py e state.py.

Na pasta Clients, incluímos: cliente.py e passenger_generator.py.

Fora destes dois diretórios, encontra-se o ficheiro main.py, onde ocorre a simulação do sistema.

Cada módulo tem responsabilidades bem definidas.

O ficheiro **client.py** define a classe **Client**, que representa cada passageiro.

Cada cliente possui:

- PID (process id): identificador único e incremental
- Classe do bilhete (first, business ou economy)
- Prioridade derivada da classe do bilhete (1, 2 ou 3)
- TTL (tempo de espera máximo até desistir)
- Estado, implementado através de um Enum (state.py), com os seguintes valores possíveis: IN_QUEUE, BOARDING, BOARDED, GAVE_UP.

- Tempos de chegada e de embarque

O ficheiro **passenger_generator.py**, que **gera passageiros** ao longo do tempo, simulando tráfego normal picos de procura (`high_demand=True`). Este gerador de clientes (executado em duas threads distintas), cria os clientes e envia-os para o servidor, que os coloca na fila de prioridades.

O ficheiro **gate.py** representa um portão de embarque com id e estado (busy ou free).

O ficheiro **agent_process.py**, onde cada agente é um **processo independente** (`multiprocessing.Process`) que recebe passageiros através de uma `task_queue`, tenta obter um portão/gate livre, faz a simulação do embarque, e no final devolve os resultados ao servidor através de uma `result_queue`.

O ficheiro **airport_server.py** atua como o núcleo do sistema, no qual o servidor:

- É responsável por manter e gerir a fila de prioridade (`PriorityQueue`)
- verifica desistências e controlar os tempos de espera (TTL expirado ou não)
- delegar passageiros para agentes de embarque (`task_queue`)
- recolhe resultados produzidos por esses agentes (`result_queue`)
- regista logs
- gere portões e agentes partilhados via `Manager().list()`

Reforçando, a fila de espera é implementada recorrendo a uma `PriorityQueue`, garantindo que os passageiros são atendidos por ordem de prioridade e, em caso de empate, pelo tempo de chegada. Além disto, os gates/portões são estruturas partilhadas entre processos, e geridas através de um `Manager().list()`, de maneira a haver garantia de que o estado de cada portão é visível e consistente entre todos os Agentes de embarque.

O ficheiro **main.py** coordena a execução, na qual inicia o servidor (aeroporto), inicia e inicializa agentes, cria threads geradoras de passageiros, aguarda o escoamento da fila e, no final da execução, recolhe todos os resultados, termina os agentes e guarda os logs num ficheiro `.txt` e encerra a execução.

Processos criados

- Múltiplos processos AgentProcess (um por agente)
- 1 processo principal (main)
- Duas threads geradoras de passageiros
- 1 thread servidor (server_loop)

Comunicação entre processos

Componente	Mecanismo	Função
Servidor → Agentes	task_queue	Envia clientes para embarcarem
Agentes → Servidor	result_queue	Devolve os resultados do embarque
Servidor ↔ Fila	PriorityQueue + Lock	Triagem por prioridade
Agentes ↔ Portões	Manager().list()	Estado partilhado dos portões
Servidor ↔ Logs	logs_lock	Escrita segura no log

Memória Partilhada

- 🚦 Lista de portões (Manager().list())
- 🚦 Fila de prioridade (PriorityQueue)
- 🚦 Filas de comunicação (Queue)
- 🚦 Logs protegidos por Lock

Decisões tomadas e dificuldades

Escolhas feitas durante a implementação e dificuldades encontradas

Uma das principais decisões tomadas incidiu no **uso de multiprocessing para representar os agentes de embarque**. Desta forma, foi possível simular uma execução paralela, pois cada agente corre num processo independente. Isto garante que vários embarques possam estar a acontecer em simultâneo, tal como acontece num aeroporto real. Assim, é possível evitar bloqueios no servidor e distribuir melhor a carga entre processos.

Outra escolha importante foi o uso de uma **PriorityQueue** para gerir a fila de passageiros/clientes. Esta fila permitiu ordenar automaticamente os passageiros de acordo com a sua prioridade (garantindo que as prioridades mais altas são atendidas primeiro) e, no caso de empate, há o desempate por arrival_time. Esta escolha simplifica a lógica do servidor e permite uma estrutura thread-safe.

Além das decisões anteriores, implementou-se um mecanismo de desistência baseada no TTL (tempo de espera / Time-To-Live) de cada passageiro. Aqui, cada passageiro tem um tempo máximo de espera e se esse tempo for excedido, o servidor altera o estado desse cliente para 'state = GAVE_UP' e regista no log. Deste modo, esta funcionalidade confere um maior realismo à simulação.

A utilização de Manager().list() para representar os Gates foi uma solução prática que permitiu que múltiplos processos pudessem aceder e modificar o estado dos portões de forma segura. No entanto, esta abordagem trouxe desafios relacionados com a sincronização, dado que vários agentes podem tentar aceder ao mesmo gate em simultâneo. Para resolver este problema, foi necessário implementar verificações repetidas e pequenas pausas (sleep) até que um portão fique disponível.

De modo a organizar melhor o código, optámos por uma **separação clara de responsabilidades** por parte dos ficheiros:

- Servidor → lógica de controlo
- Agentes → embarque
- Geradores → criação de passageiros
- Cliente → dados e comportamento

Por fim, determinámos a existência de **logs detalhados**, onde cada evento é registado com timestamp de: ARRIVAL, DISPATCH, BOARDED ou GAVE UP.

O ficheiro de logs resultante confirma que o funcionamento correto do sistema, no entanto, também é possível verificar os logs durante a execução/simulação do programa pelo terminal (executando o ficheiro na linha de comandos com '**python3 main.py**').

Dificuldades Encontradas

1. Sincronização entre processos

No que diz respeito às dificuldades encontradas, destacamos a necessidade de uma **correta coordenação de threads e processos**.

Foi necessário garantir que:

- os dois agentes não ocupassem o mesmo portão;
- a fila não fosse acedida simultaneamente;
- os logs fossem escritos sem condições de corrida.

Para solucionar estas dificuldades, utilizámos: Lock, Manager().list(), Queue.

2. Gestão de desistências

A verificação de TTL exigiu: a remoção temporária de todos os elementos da fila, a filtração de clientes desistentes e a reintrodução dos restantes.

3. Coordenação entre servidor e agentes

O servidor não podia bloquear enquanto agentes trabalhavam. Para solucionar este problema, fizemos com que os agentes corressem em processos separados e com que o servidor recolhesse resultados de forma a não bloquear enquanto aguardava resultados dos agentes (`get_nowait()`), continuando a processar a fila.

4. Evitar starvation

Com muitos passageiros de prioridade baixa, foi necessário garantir que: o TTL fosse suficientemente grande, e que a fila fosse processada rapidamente.

5. Concorrência entre threads e processos

Misturar threads (servidor, geradores) com processos (agentes) exigiu cuidado extra na partilha de dados.

6. Terminação de agentes

A gestão da terminação dos agentes também exigiu algum cuidado. Foi necessário enviar um valor `None` para cada agente, para indicar que o processo devia terminar, e garantir que todos os agentes eram corretamente encerrados antes de gravar os logs finais.

Propostas para futuras melhorias

Sugestões para aprimorar o sistema, como a adição de novas funcionalidades ou otimizações

Uma das melhorias mais interessantes seria a criação de uma **interface gráfica** que permitisse: visualizar em tempo real o estado da fila, os portões ocupados, os agentes ativos e os passageiros em processo de embarque. Isto tornaria a simulação mais intuitiva e iria permitir uma análise visual do comportamento do sistema.

Poderia também ser implementado posteriormente um sistema de **balanceamento inteligente de portões**, em que os agentes pudessem escolher o portão menos utilizado ou aquele que estivesse livre há mais tempo, em vez de simplesmente escolherem o primeiro portão disponível (possivelmente melhorado através de load balancing ou da priorização por tipo de passageiro).

Finalmente, poderia ser possível introduzir **prioridades dinâmicas**, em que passageiros que esperassem demasiado tempo pudessem subir de prioridade, reduzindo o risco de starvation para passageiros de classe económica, ou até a perda de prioridade (ou descida de prioridade) se chegassem mais tarde do que o suposto ao aeroporto.